

Problem 1: Geometric Image Transformation

Part 1: Implementing Basic and Complex Transformations

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def apply_translation(image, tx, ty):
    rows, cols = image.shape[:2]
    translation_matrix = np.float32([[1, 0, tx], [0, 1, ty]])
    translated_image = cv2.warpAffine(image, translation_matrix,
    (cols, rows))
    return translated_image

def apply_rotation(image, theta):
    rows, cols = image.shape[:2]
    center = (cols / 2, rows / 2)
    rotation_matrix = cv2.getRotationMatrix2D(center, theta, 1)
    rotated_image = cv2.warpAffine(image, rotation_matrix, (cols,
    rows))
    return rotated_image

def apply_shearing(image, shx, shy):
    rows, cols = image.shape[:2]
    shearing_matrix = np.float32([[1, shx, 0], [shy, 1, 0]])
    sheared_image = cv2.warpAffine(image, shearing_matrix, (cols,
    rows))
    return sheared_image

def apply_scaling_and_fill(image, sx, sy):
    rows, cols = image.shape[:2]
    scaled_image = cv2.resize(image, None, fx=sx, fy=sy,
    interpolation=cv2.INTER_LINEAR)

    # Create a black image with the original size
    black_image = np.zeros_like(image)

    # Compute the starting coordinates (top-left) to place the scaled
    image at the center
    start_row = (rows - scaled_image.shape[0]) // 2
    start_col = (cols - scaled_image.shape[1]) // 2

    # Place the scaled image on the black image
    black_image[start_row:start_row+scaled_image.shape[0],
    start_col:start_col+scaled_image.shape[1]] = scaled_image

    return black_image
```

```
# Load image
image = cv2.imread('./geisel_128.jpg', cv2.IMREAD_GRAYSCALE)

# Apply transformations
translated_image = apply_translation(image, 10, 10)
rotated_image = apply_rotation(image, 45)
sheared_image = apply_shearing(image, 0.5, 0.2)
scaled_image = apply_scaling_and_fill(image, 1/2, 1/2)

# Create subplots
fig, axs = plt.subplots(2, 3, figsize=(15, 10))

# Display images
axs[0, 0].imshow(image, cmap='gray')
axs[0, 0].set_title('Original Image')

axs[0, 1].imshow(translated_image, cmap='gray')
axs[0, 1].set_title('Translated Image')

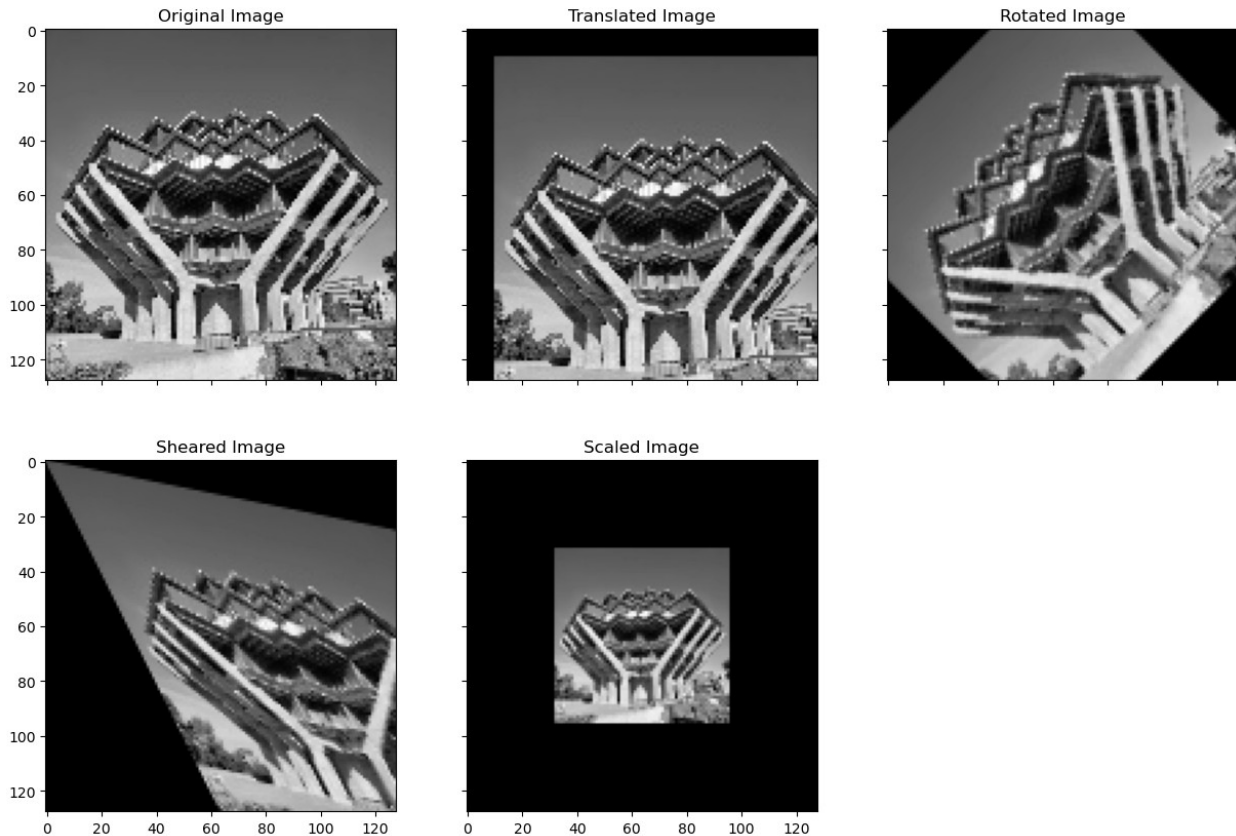
axs[0, 2].imshow(rotated_image, cmap='gray')
axs[0, 2].set_title('Rotated Image')

axs[1, 0].imshow(sheared_image, cmap='gray')
axs[1, 0].set_title('Sheared Image')

axs[1, 1].imshow(scaled_image, cmap='gray')
axs[1, 1].set_title('Scaled Image')

fig.delaxes(axs[1, 2])
for ax in axs.flat:
    ax.label_outer()

plt.show()
```



Part 2: Transformation with Interpolation

1. Scaling with Integer Coordinates (Easy Case)

```
def nearest_neighbor_interpolation(image, new_size):
    if len(image.shape) == 3:
        rows, cols, channels = image.shape
    else:
        rows, cols = image.shape
        channels = 1
        image = image[:, :, np.newaxis]

    new_image = np.zeros((new_size[0], new_size[1], channels),
dtype=image.dtype)

    row_ratio, col_ratio = rows / new_size[0], cols / new_size[1]

    for r in range(new_size[0]):
        for c in range(new_size[1]):
            old_r, old_c = int(r * row_ratio), int(c * col_ratio)
            new_image[r, c] = image[old_r, old_c]

    return new_image.squeeze()
```

```

def bilinear_interpolation(image, new_size):
    if len(image.shape) == 3:
        rows, cols, channels = image.shape
    else:
        rows, cols = image.shape
        channels = 1
        image = image[:, :, np.newaxis]

    new_image = np.zeros((new_size[0], new_size[1], channels),
dtype=image.dtype)

    row_ratio, col_ratio = rows / new_size[0], cols / new_size[1]

    for r in range(new_size[0]):
        for c in range(new_size[1]):
            r_low, c_low = int(r * row_ratio), int(c * col_ratio)
            r_high, c_high = min(r_low + 1, rows - 1), min(c_low + 1,
cols - 1)

            r_frac, c_frac = r * row_ratio - r_low, c * col_ratio -
c_low

            for ch in range(channels):
                top_left = (1 - r_frac) * (1 - c_frac) * image[r_low,
c_low, ch]
                top_right = (1 - r_frac) * c_frac * image[r_low,
c_high, ch]
                bottom_left = r_frac * (1 - c_frac) * image[r_high,
c_low, ch]
                bottom_right = r_frac * c_frac * image[r_high, c_high,
ch]

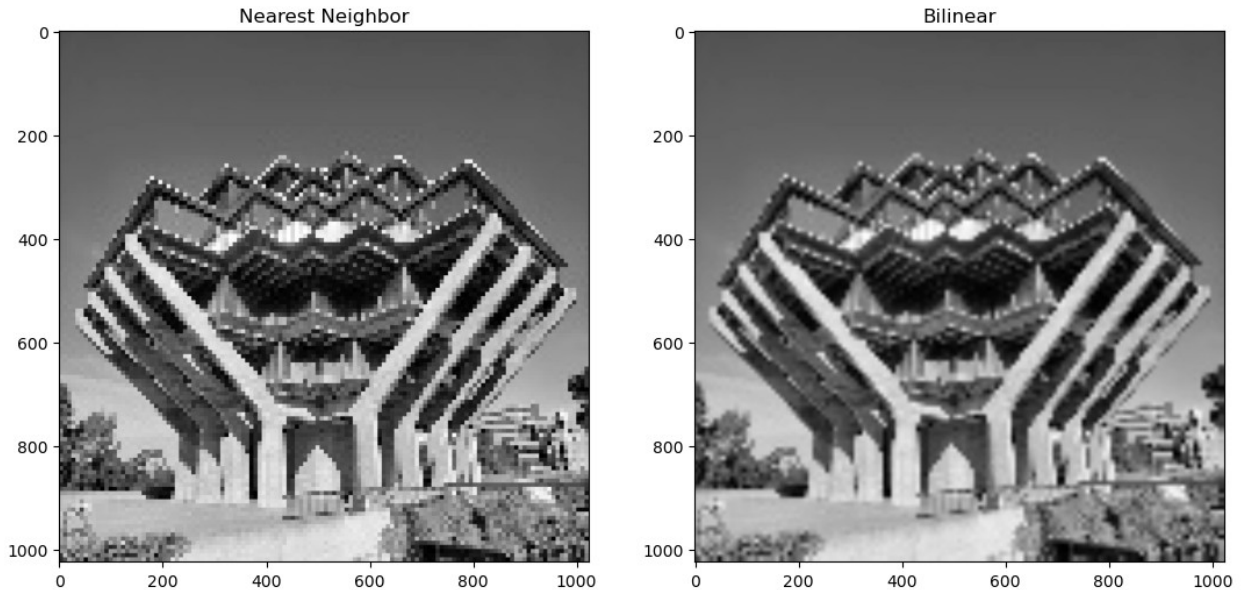
                new_image[r, c, ch] = top_left + top_right +
bottom_left + bottom_right

    return new_image.squeeze()

# Apply scaling
nearest = nearest_neighbor_interpolation(image, (1024, 1024))
bilinear = bilinear_interpolation(image, (1024, 1024))

plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(nearest, cmap='gray')
plt.title("Nearest Neighbor")
plt.subplot(1, 3, 2)
plt.imshow(bilinear, cmap='gray')
plt.title("Bilinear")
plt.show()

```



2. Scaling with Non-Integer Coordinates (Challenging Case)

```
def apply_scaling_non_integer_coords(image, scaling_factor):
    rows, cols = image.shape[:2]
    new_size = (int(cols * scaling_factor), int(rows *
scaling_factor))

    # Create empty images for the results
    nearest_image = np.zeros(new_size, dtype=image.dtype)
    bilinear_image = np.zeros(new_size, dtype=image.dtype)

    # Compute the inverse scaling factor
    inv_scale = 1 / scaling_factor

    # Fill each pixel in the new images
    for r in range(new_size[1]):
        for c in range(new_size[0]):
            # Compute the corresponding location in the original image
            old_r, old_c = r * inv_scale, c * inv_scale

            # Nearest-neighbor interpolation
            nearest_r, nearest_c = min(int(round(old_r)), rows - 1),
min(int(round(old_c)), cols - 1)
            nearest_image[r, c] = image[nearest_r, nearest_c]

            # Bilinear interpolation
            r_low, c_low = int(old_r), int(old_c)
            r_high, c_high = min(r_low + 1, rows - 1), min(c_low + 1,
cols - 1)
            r_frac, c_frac = old_r - r_low, old_c - c_low
            top_left = (1 - r_frac) * (1 - c_frac) * image[r_low,
c_low]
```

```

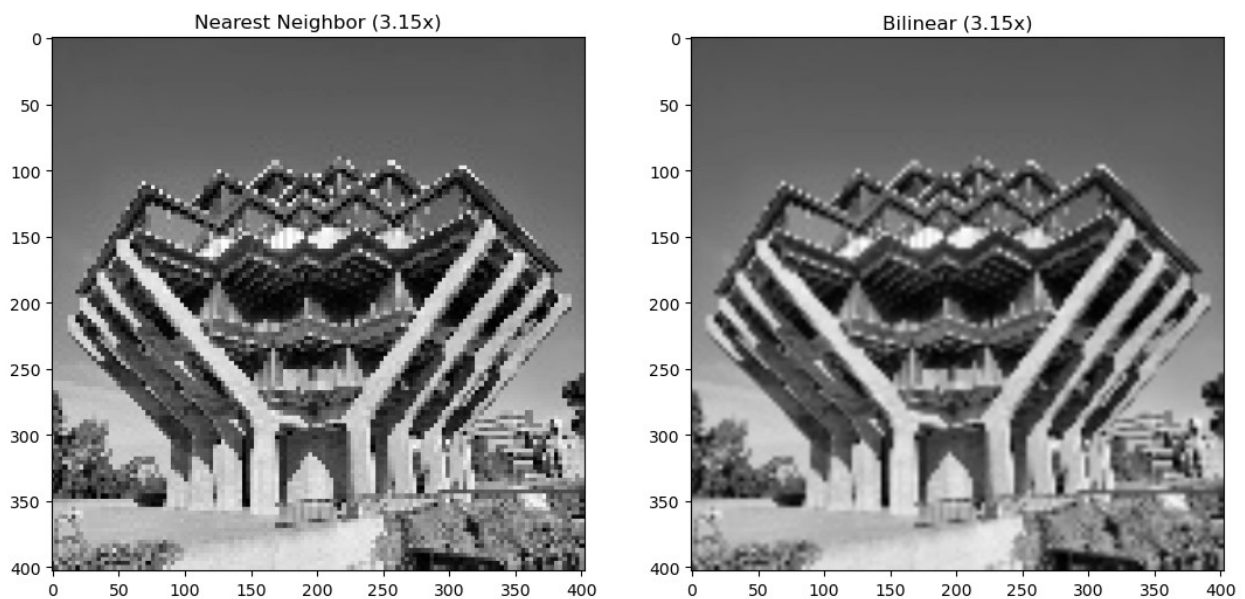
        top_right = (1 - r_frac) * c_frac * image[r_low, c_high]
        bottom_left = r_frac * (1 - c_frac) * image[r_high, c_low]
        bottom_right = r_frac * c_frac * image[r_high, c_high]
        bilinear_image[r, c] = top_left + top_right + bottom_left
+ bottom_right

    return nearest_image, bilinear_image

nearest, bilinear = apply_scaling_non_integer_coords(image, 3.15)

plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(nearest, cmap='gray')
plt.title("Nearest Neighbor (3.15x)")
plt.subplot(1, 3, 2)
plt.imshow(bilinear, cmap='gray')
plt.title("Bilinear (3.15x)")
plt.show()

```



Problem 2: Image Decimation

```

def apply_averaging_filter(image):
    kernel = np.ones((3, 3), np.float32) / 9
    filtered_image = cv2.filter2D(image, -1, kernel)
    return filtered_image

image = cv2.imread('./spatial_freq.jpg', cv2.IMREAD_GRAYSCALE)
filtered_image = apply_averaging_filter(image)

```

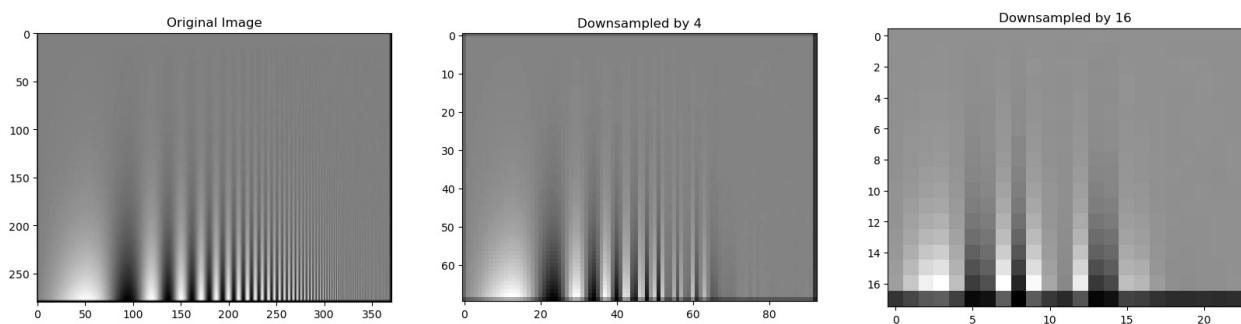
```

# Downsample by factor of 4
downsampled_4 = cv2.resize(filtered_image, None, fx=0.25, fy=0.25)
# Downsample by factor of 16
downsampled_16 = cv2.resize(filtered_image, None, fx=1/16, fy=1/16)

plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.subplot(1, 3, 2)
plt.imshow(downsampled_4, cmap='gray')
plt.title("Downsampled by 4")
plt.subplot(1, 3, 3)
plt.imshow(downsampled_16, cmap='gray')
plt.title("Downsampled by 16")

plt.show()

```



Observations: Aliasing. This results in the columns appear to be more uniformly distributed than they actually are in the original image.

Problem 3: Interpolation with a Custom Filter

```

def apply_custom_filter(image):
    kernel = np.array([[1, 2, 3, 2, 1],
                       [2, 3, 4, 3, 2],
                       [3, 4, 5, 4, 3],
                       [2, 3, 4, 3, 2],
                       [1, 2, 3, 2, 1]], np.float32) / 25

    filtered_image = cv2.filter2D(image, -1, kernel)
    return filtered_image

# Upsample by factor of 2
upsampled_image = cv2.resize(image, None, fx=2, fy=2)

```



```

# Apply the custom filter to the upsampled image
filtered_upsampled_image = apply_custom_filter(upsampled_image)

# Downsample back to the original size
downsampled_back = cv2.resize(filtered_upsampled_image,
image.shape[:-1])

# Create a black canvas of the size of the upsampled image
canvas = np.full_like(upsampled_image, 255)

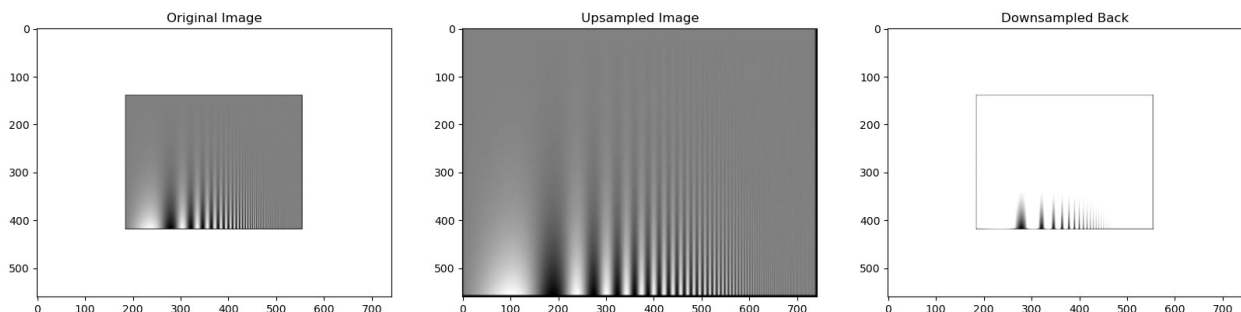
# Calculate the position to place the original and downsampled images
start_row, start_col = (upsampled_image.shape[0] - image.shape[0]) //
2, (upsampled_image.shape[1] - image.shape[1]) // 2

# Place the original and downsampled images in the center of the
canvases
canvas_with_image = canvas.copy()
canvas_with_image[start_row:start_row+image.shape[0],
start_col:start_col+image.shape[1]] = image

canvas_with_downsampled = canvas.copy()
canvas_with_downsampled[start_row:start_row+downsampled_back.shape[0],
start_col:start_col+downsampled_back.shape[1]] = downsampled_back

plt.figure(figsize=(20, 10))
plt.subplot(1, 3, 1)
plt.imshow(canvas_with_image, cmap='gray')
plt.title("Original Image")
plt.subplot(1, 3, 2)
plt.imshow(upsampled_image, cmap='gray')
plt.title("Upsampled Image")
plt.subplot(1, 3, 3)
plt.imshow(canvas_with_downsampled, cmap='gray')
plt.title("Downsampled Back")
plt.show()

```



Observation: The filter is not half-band, therefore does not preserve the signal after a series of up and down sampling